4. Yara Rules

Task 1 - What is Yara?

Yara can identify information based on both binary and textual patterns, such as hexadecimal and strings contained within a file. Rules are used to label these patterns. For example, Yara rules are frequently written to determine if a file is malicious or not, based upon the features - or patterns - it presents. Strings are a fundamental component of programming languages. Applications use strings to store data such as text. Malware uses strings to store textual data.

Answer the questions below

> What is the name of the base-16 numbering system that Yara can detect?
> hexadecimal

> Would the text "Enter your Name" be a string in an application? Yay

Task 2 - Introduction to Yara Rules

The proprietary language that Yara uses for rules is fairly trivial to pick up, but hard to master. This is because your rule is only as effective as your understanding of the patterns you want to search for. Using a Yara rule is simple. Every yara command requires two arguments to be valid, these are:

1. The rule file we create
2. Name of file, directory, or process ID to use the rule for.

Every rule must have a name and condition. For example, if we wanted to use "myrule.yar" on directory "some directory", we would use the following command:

`yara myrule.yar somedirectory`

Note that **.yar** is the standard file extension for all Yara rules. We'll make one of the most basic rules you can make below.

1. Make a file named "**somefile**" via `touch somefile`

2. Create a new file and name it "**myfirstrule.yar**" like below:

Creating a file named somefile

```
cmnatic@thm:~$ touch somefile
```

Creating a file named myfirstrule.yar

```
cmnatic@thm touch myfirstrule.yar
```

3. Open the "myfirstrule.yar" using a text editor such as `nano` and input the snippet below and save the file:

```
rule examplerule {
        condition: true
}
```

Inputting our first snippet into "myfirstrule.yar" using nano

```
cmnatic@thm nano myfirstrule.yar   GNU nano 4.8 myfirstrule.y
ar
rule examplerule {
        condition: true
}
```

The **name** of the rule in this snippet is `examplerule`, where we have one condition - in this case, the **condition** is `condition`.

As previously discussed, every rule requires both a name and a condition to be valid. This rule has satisfied those two requirements.

Simply, the rule we have made checks to see if the file/directory/PID that we specify exists via `condition: true`. If the file does exist, we are given the output of `examplerule`

Task 3 - Expanding on Yara Rules

Yara has a few conditions,
Meta: This section of a Yara rule is reserved for descriptive information by the author of the rule. For example, you can use desc , short for description, to summarise what your rule checks for. Anything within this section does not

influence the rule itself. Similar to commenting code, it is useful to summarise your rule.

Strings: Remember our discussion about strings in Task 2? Well, here we go. You can use strings to search for specific text or hexadecimal in files or programs. For example, say we wanted to search a directory for all files containing "Hello World!", we would create a rule such as below:

```
rule helloworld_checker{
    strings:
        $hello_world = "Hello World!"
}
```

We define the keyword Strings where the string that we want to search, i.e., "Hello World!" is stored within the variable $hello_world Of course, we need a condition here to make the rule valid. In this example, to make this string the condition, we need to use the variable's name.

```
rule helloworld_checker{
    strings:
        $hello_world = "Hello World!"

    condition:
        $hello_world
}
```

Essentially, if any file has the string "Hello World!" then the rule will match. However, this is literally saying that it will only match if "Hello World!" is found and will not match if "*hello world*" or "*HELLO WORLD*." To solve this, the condition any of them allows multiple strings to be searched for, like below:

```
rule helloworld_checker{
    strings:
        $hello_world = "Hello World!"
        $hello_world_lowercase = "hello world"
        $hello_world_uppercase = "HELLO WORLD"
```

```
    condition:
        any of them
}
```

Now, any file with the strings of:

1. Hello World!

2. hello world

3. HELLO WORLD

Will now trigger the rule.

Conditions

We have already used the `true` and `any of them` condition. Much like regular programming, you can use operators such as:

- **<=** (less than or equal to)
- **>=** (more than or equal to)
- **!=** (not equal to)

For example, the rule below would do the following:

```
rule helloworld_checker{
    strings:
        $hello_world = "Hello World!"

    condition:
        #hello_world <= 10
}
```

The rule will now:

1. Look for the "Hello World!" string

2. Only say the rule matches if there are less than or equal to ten occurrences of the "Hello World!" string

Combining keywords

Moreover, you can use keywords such as:

- and
- not
- or

To combine multiple conditions. Say if you wanted to check if a file has a string and is of a certain size (in this example, the sample file we are checking is

less than

<10 kb and has "Hello World!" you can use a rule like below:

```
rule helloworld_checker{
    strings:
        $hello_world = "Hello World!"

        condition:
            $hello_world and filesize < 10KB
}
```

The rule will only match if both conditions are true. To illustrate: below, the rule we created, in this case, did not match because although the file has "Hello World!", it has a file size larger than 10KB:

Yara failing to match the file mytextfile because it is larger than 10kb

```
cmnatic@thm:~$ <output intentionally left blank>
```

However, the rule matched this time because the file has both "Hello World!" and a file size of less than 10KB.

Yara successfully matching the file mytextfile because it has "Hello World" and a file size of less than 10KB

```
cmnatic@thm:~$ yara myfirstrule.yar mytextfile.txt
helloworld_textfile_checker mytextfile.txt
```

Remembering that the text within the red box is the name of our rule, and the text within the green is the matched file.

Task 4 - Yara Modules

Cuckoo: Cuckoo Sandbox is an automated malware analysis environment. This module allows you to generate Yara rules based upon the behaviours discovered from Cuckoo Sandbox. As this environment executes malware, you can create rules on specific behaviours such as runtime strings and the like.

Python PE: Python's PE module allows you to create Yara rules from the various sections and elements of the Windows Portable Executable (PE) structure.

Explaining this structure is out of scope as it is covered in my malware introductory room. However, this structure is the standard formatting of all executables and DLL files on windows. Including the programming libraries that are used.

Examining a PE file's contents is an essential technique in malware analysis; this is because behaviours such as cryptography or worming can be largely identified without reverse engineering or execution of the sample.

Task 5 - Other tools and Yara

LOKI : LOKI is a free open-source IOC (*Indicator of Compromise*) scanner created/written by Florian Roth. Based on the GitHub page, detection is based on 4 methods:

1. File Name IOC Check
2. Yara Rule Check **(we are here)**
3. Hash Check
4. C2 Back Connect Check

THOR: THOR *Lite* is Florian's newest multi-platform IOC AND YARA scanner. There are precompiled versions for Windows, Linux, and macOS. A nice feature with THOR Lite is its scan throttling to limit exhausting CPU resources.

FENRIR: This is the 3rd tool(opens in new tab) created by Neo23×0 (Florian Roth). You guessed it; the previous 2 are named above. The updated version was created to address the issue from its predecessors, where requirements must be

met for them to function. Fenrir is a bash script; it will run on any system capable of running bash (nowadays even Windows).

YAYA: YAYA was created by the EFF(opens in new tab) (*Electronic Frontier Foundation*) and released in September 2020. Based on their website, "*YAYA is a new open-source tool to help researchers manage multiple YARA rule repositories. YAYA starts by importing a set of high-quality YARA rules and then lets researchers add their own rules, disable specific rulesets, and run scans of files.*"

Task 6 - Using LOKI and its Yara rule set

Using LOKI: As a security analyst, you may need to research various threat intelligence reports, blog postings, etc. and gather information on the latest tactics and techniques used in the wild, past or present. Typically in these readings, IOCs (hashes, IP addresses, domain names, etc.) will be shared so rules can be created to detect these threats in your environment, along with Yara rules.

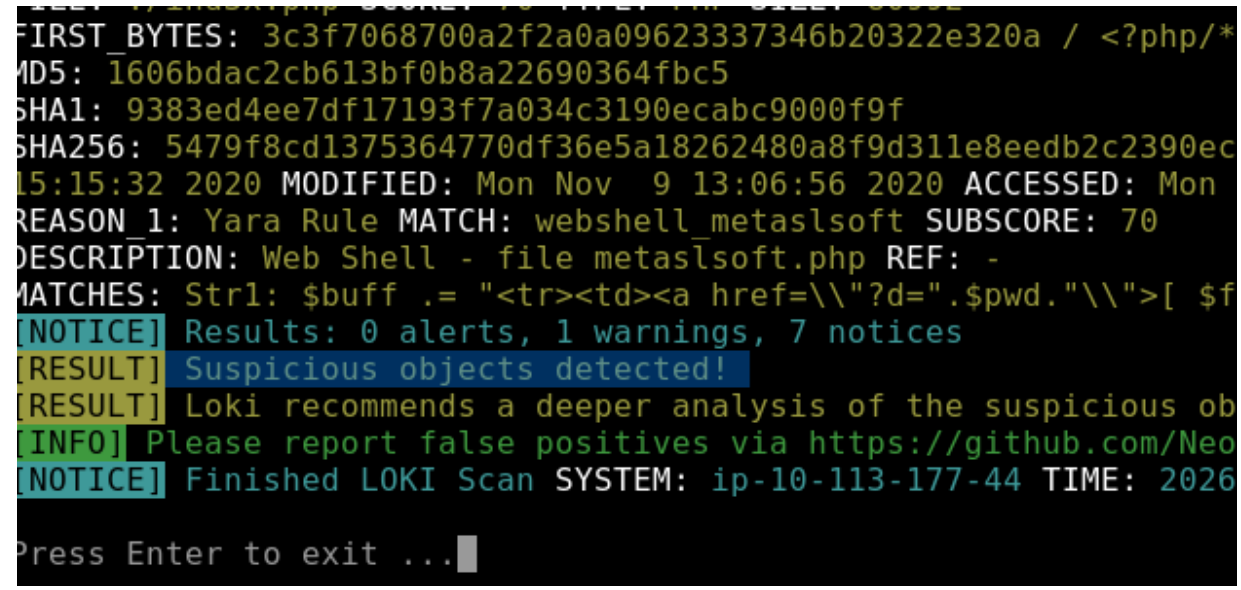
Scenario: You are the security analyst for a mid-size law firm. A co-worker discovered suspicious files on a web server within your organization. These files were discovered while performing updates to the corporate website. The files have been copied to your machine for analysis. The files are located in the `suspicious-files` directory. Use Loki to answer the questions below.

Answer the questions below

> Scan file 1. Does Loki detect this file as suspicious/malicious or benign?

Suspicious (python ../../tools/Loki/loki.py-p . runs the **Loki scanner**, an open-source **IOC (Indicator of Compromise) and malware detection tool** commonly used in DFIR (Digital Forensics and Incident Response) and threat hunting.

```
cmnatic@ip-10-113-177-44:~$ ls
go  myfirstrule.yar  suspicious-files  tools  yara-python
cmnatic@ip-10-113-177-44:~$ cd suspicious-files
cmnatic@ip-10-113-177-44:~/suspicious-files$ ls
file1  file2
cmnatic@ip-10-113-177-44:~/suspicious-files$ cd file1
cmnatic@ip-10-113-177-44:~/suspicious-files/file1$ ls
ind3x.php  loki_ip-10-10-32-16_2025-07-27_15-42-59.log
cmnatic@ip-10-113-177-44:~/suspicious-files/file1$ python ../../tools/Loki/loki.py -p .
```



python: Runs the Python interpreter.

```
../../tools/Loki/loki.py
```

Executes the `loki.py` script located in:

```
../../tools/Loki/
```

The `../` means "go up one directory level," so `../../` means "go up two levels."

The `-p` (path) option tells Loki which directory to scan.

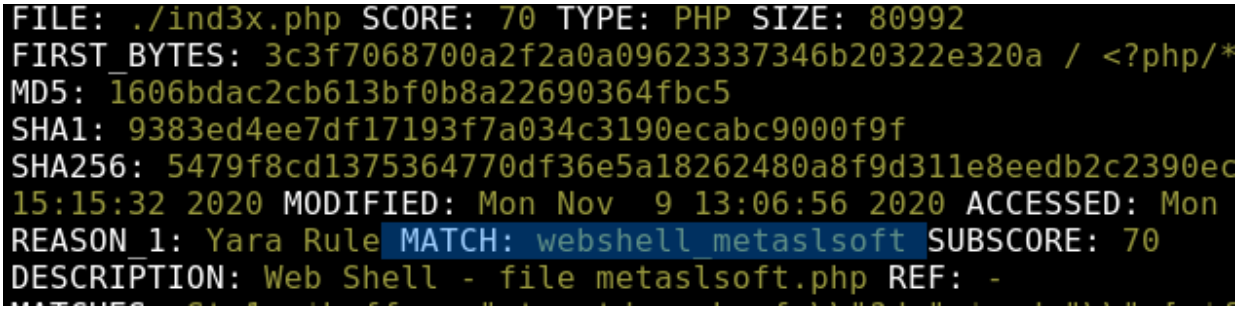
The dot (`.`) means: current directory

So Loki will recursively scan the current folder and its subfolders for:

- Malware signatures
- Suspicious files
- Indicators of Compromise (IOCs)
- YARA rule matches
- Known malicious hashes
- Suspicious processes and artifacts

> What Yara rule did it match on? webshell_metaslsoft

What does Loki classify this file as? Web Shell



A **web shell** is a malicious script placed on a web server that allows an attacker to remotely execute commands through a web browser.

Purpose:

- Gain unauthorized access to a server
- Execute system commands
- Upload/download files
- Maintain persistence after a compromise

Common file types:

- `.php`
- `.aspx`

- `.jsp`

Example attack flow:

1. Attacker exploits a vulnerable website.
2. Uploads a web shell.
3. Accesses it via a URL.
4. Controls the server remotely.

a web shell is a **backdoor on a web server that lets attackers control the system remotely**.

> Based on the output, what string within the Yara rule did it match on? Str1

```
FILE: ./ind3x.php SCORE: 70 TYPE: PHP SIZE: 80992
FIRST_BYTES: 3c3f7068700a2f2a0a09623337346b20322e320a / <?php/*b374k 2.2
MD5: 1606bdac2cb613bf0b8a22690364fbc5
SHA1: 9383ed4ee7df17193f7a034c3190ecabc9000f9f
SHA256: 5479f8cd1375364770df36e5a18262480a8f9d311e8eedb2c2390ecb233852ad CRE
15:15:32 2020 MODIFIED: Mon Nov  9 13:06:56 2020 ACCESSED: Mon Jul 13 07:27:
REASON_1: Yara Rule MATCH: webshell_metaslsoft SUBSCORE: 70
DESCRIPTION: Web Shell - file metaslsoft.php REF: -
MATCHES: Str1: $buff .= "<tr><td><a href=\\"?d=".$pwd."\\">[ $folder ]</a></
```

> What is the name and version of this hack tool? b374k 2.2

Inspect the actual Yara file that flagged file 1. Within this rule, how many strings are there to flag this file? 1

```
FILE: ./ind3x.php SCORE: 70 TYPE: PHP SIZE: 80992
FIRST_BYTES: 3c3f7068700a2f2a0a09623337346b20322e320a / <?php/*b374k 2.2
MD5: 1606bdac2cb613bf0b8a22690364fbc5
SHA1: 9383ed4ee7df17193f7a034c3190ecabc9000f9f
SHA256: 5479f8cd1375364770df36e5a18262480a8f9d311e8eedb2c2390ecb233852ad CR
15:15:32 2020 MODIFIED: Mon Nov  9 13:06:56 2020 ACCESSED: Mon Jul 13 07:27
REASON_1: Yara Rule MATCH: webshell_metaslsoft SUBSCORE: 70
DESCRIPTION: Web Shell - file metaslsoft.php REF: -
MATCHES: Str1: $buff .= "<tr><td><a href=\\"?d=".$pwd."\\">[ $folder ]</a><
```

> Scan file 2. Does Loki detect this file as suspicious/malicious or benign? Benign

```
[NOTICE] Running plugin PluginWMI
[NOTICE] Finished running plugin PluginWMI
[INFO] Scanning . ...
[NOTICE] Results: 0 alerts, 0 warnings, 7 notices
[RESULT] SYSTEM SEEMS TO BE CLEAN.
[INFO] Please report false positives via https://github.com/Neo23x0/signatu
[NOTICE] Finished LOKI Scan SYSTEM: ip-10-113-177-44 TIME: 20260713T07:45:2
```

> Inspect file 2. What is the name and version of this web shell? b374k 3.2.3

more 1ndex.php

```
<?php
/*
        b374k shell 3.2.3
        Jayalah Indonesiaku
        (c)2014
        https://github.com/b374k/b374k

*/
```

Task 7 - Creating Yara rules with yarGen

We need to create a Yara rule to detect this specific web shell in our environment. Typically this is what is done in the case of an incident, which is an event that affects/impacts the organization in a negative fashion. We can manually open the file and attempt to sift through lines upon lines of code to find possible strings that can be used in our newly created Yara rule.

python3 yarGen.py -m /home/cmnatic/suspicious-files/file2 --excludegood -o /home/cmnatic/suspicious-files/file2.yar (generates a YARA detection rule from a suspicious file, excluding common legitimate strings to reduce false positives, and saves the rule as `file2.yar`.)

- `m` → **Malware/sample file input**: specifies the suspicious file that yarGen will analyze and use to create the YARA rule. It means **"generate a rule based on this file."**
- `o` → **Output file**: specifies where to save the generated YARA rule. It means **"save the generated rule as file2.yar."**

In short:

- `m` **= input malware file**

- `o` **= output YARA rule file**

yarGen - Creates YARA rules automatically
YARA - Uses rules to scan and detect malware

Part	Explanation
`python3`	Runs the script using Python 3
`yarGen.py`	The yarGen tool that automatically generates YARA rules
`-m /home/cmnatic/suspicious-files/file2`	Specifies the **malware/suspicious file** to analyze and generate a rule for
`--excludegood`	Excludes strings that are commonly found in legitimate files to reduce false positives. It removes common strings
`-o /home/cmnatic/suspicious-files/file2.yar`	Saves the generated YARA rule into the file `file2.yar`

Answer the questions below

> From within the root of the suspicious files directory, what command would you run to test Yara and your Yara rule against file 2?

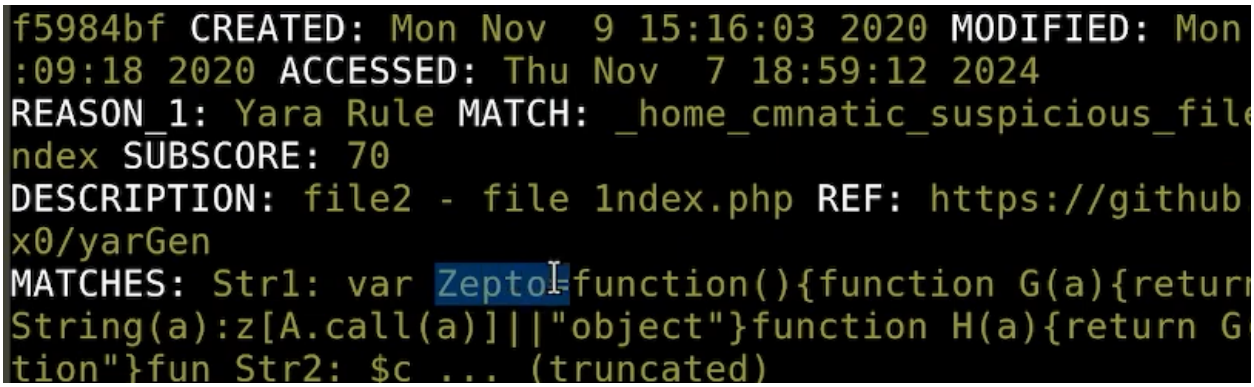
yara file2.yar file2/1ndex.php

> Did Yara rule flag file 2? Yay

> Test the Yara rule with Loki, does it flag file 2? Yay

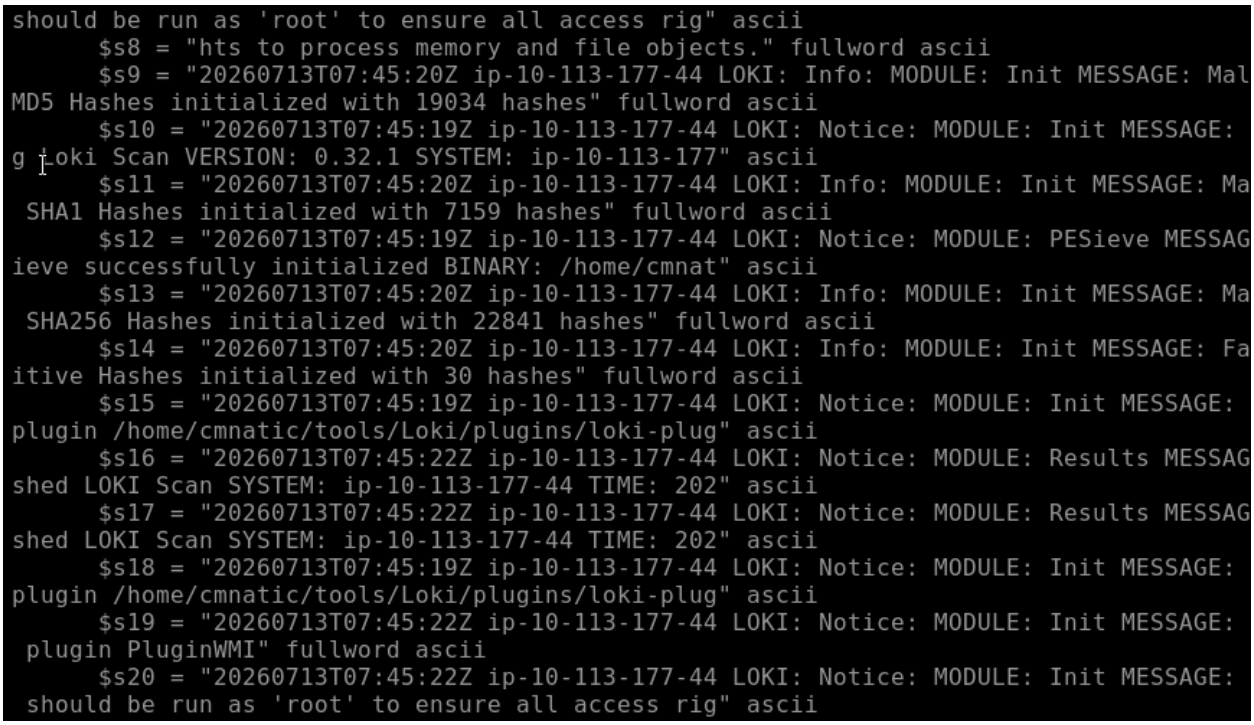
> What is the name of the variable for the string that it matched on? Zepto

python ~/tools/Loki/loki.ph -p ~/suspicious-files/file2



> Inspect the Yara rule, how many strings were generated?

20 (open rule with text editor, check strings)



> One of the conditions to match on the Yara rule specifies file size. The file has to be less than what amount?

700KB (at the end of the rule)

Task 8 - Valhalla

Valhalla: Valhalla is an online Yara feed created and hosted by Nextron-Systems(opens in new tab) (erm, Florian Roth). By now, you should be aware of the ridiculous amount of time and energy Florian has dedicated to creating these tools for the community. Maybe we should have just called this the Florian Roth room. (lol)

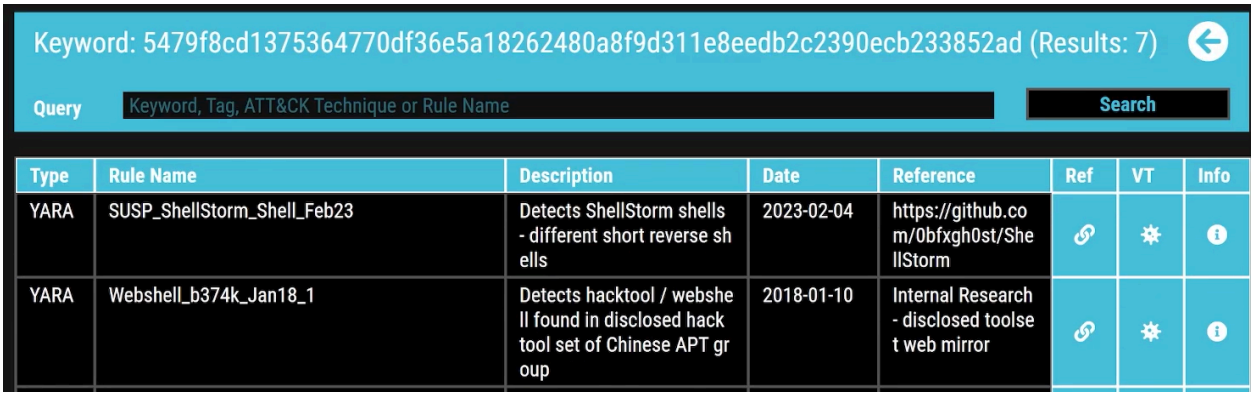
Answer the questions below

> Enter the SHA256 hash of file 1 into Valhalla. Is this file attributed to an APT group?

Yay

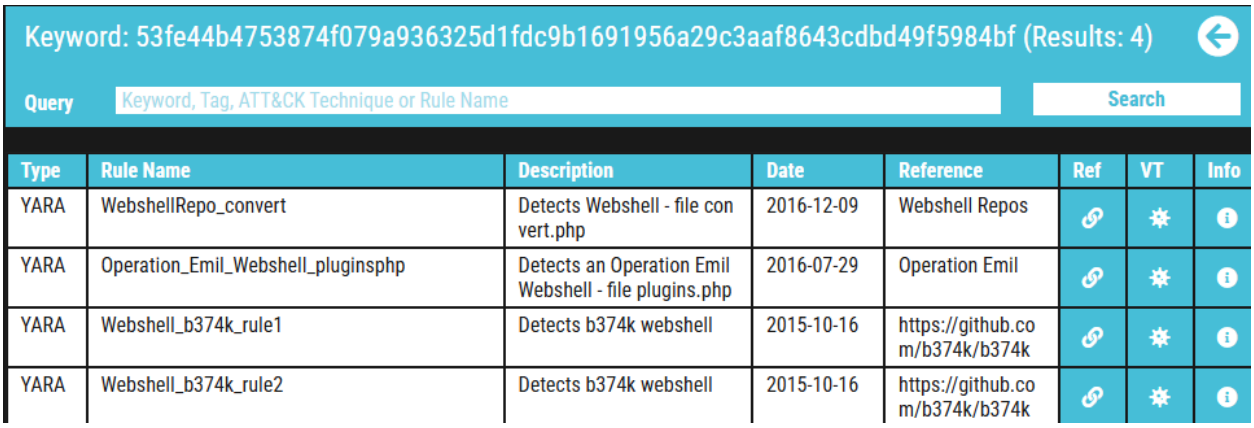
```
cmnatic@ip-10-113-177-44:~/suspicious-files/file1$ ls
ind3x.php                                  loki_ip-10-113-177-44_2026-07-13_07-27-38.log
loki_ip-10-10-32-16_2025-07-27_15-42-59.log
cmnatic@ip-10-113-177-44:~/suspicious-files/file1$ sha256sum ind3x.php
5479f8cd1375364770df36e5a18262480a8f9d311e8eedb2c2390ecb233852ad  ind3x.php
```

Valhalla YARA Rules - Valhalla



> Do the same for file 2. What is the name of the first Yara rule to detect file 2?

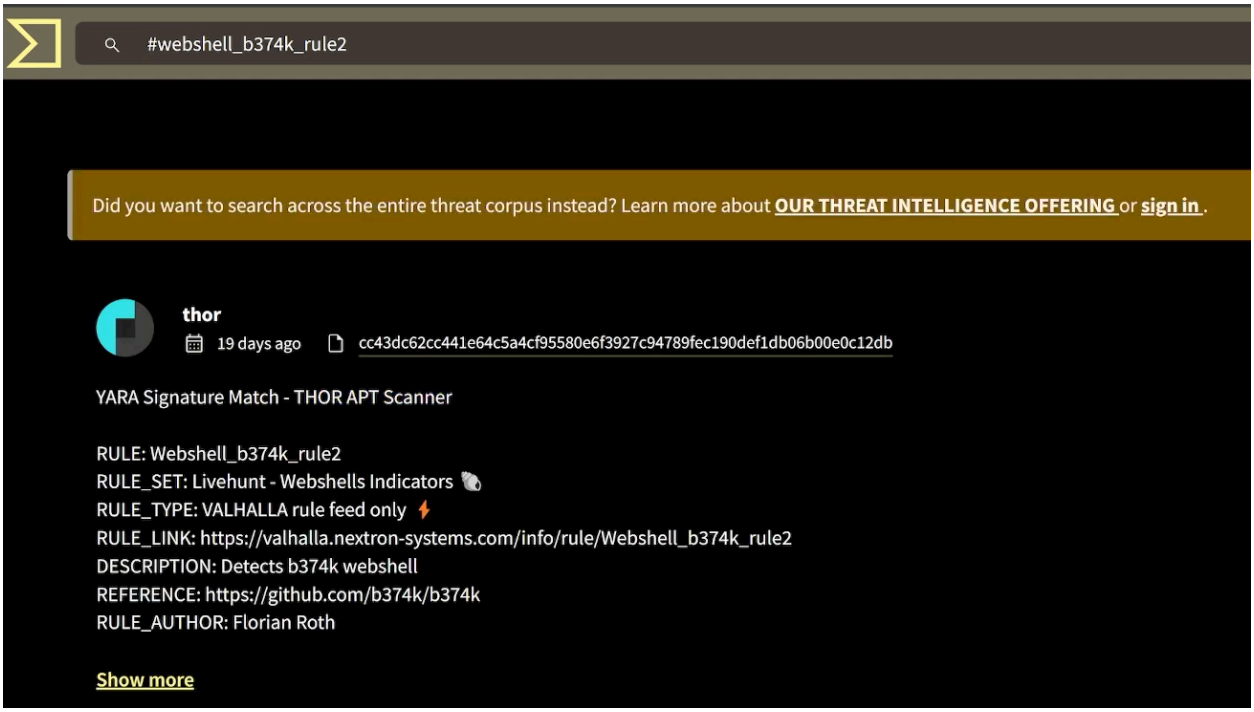
Webshell_b374k_rule1



Type	Rule Name	Description	Date	Reference	Ref	VT	Info
YARA	WebshellRepo_convert	Detects Webshell - file convert.php	2016-12-09	Webshell Repos			
YARA	Operation_Emil_Webshell_pluginsphp	Detects an Operation Emil Webshell - file plugins.php	2016-07-29	Operation Emil			
YARA	Webshell_b374k_rule1	Detects b374k webshell	2015-10-16	https://github.com/b374k/b374k			
YARA	Webshell_b374k_rule2	Detects b374k webshell	2015-10-16	https://github.com/b374k/b374k			

> Examine the information for file 2 from Virus Total (VT). The Yara Signature Match is from what scanner?

THOR APT Scanner



> Enter the SHA256 hash of file 2 into Virus Total. Did every AV detect this as malicious?

Nay

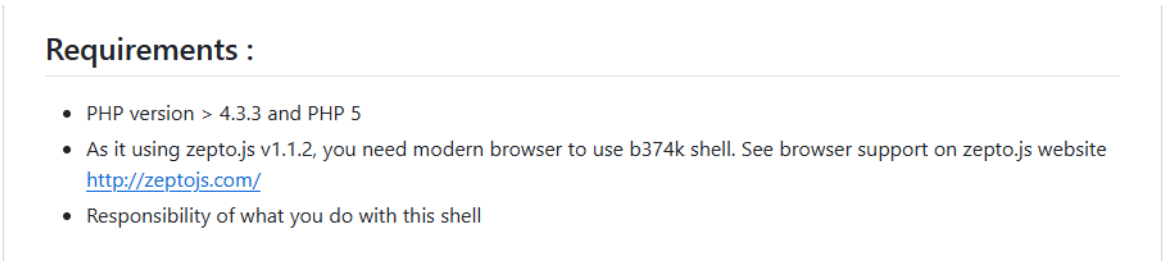
Besides .PHP, what other extension is recorded for this file? EXE



> What JavaScript library is used by file 2? Zepto

click the details:





> Is this Yara rule in the default Yara file Loki uses to detect these type of hack tools? Nay

Task 9 - Conclusion

In this room, we explored Yara, how to use Yara, and manually created basic Yara rules. We also explored various open-source tools to hit the ground running that utilizes Yara rules to detect evil on endpoints.

By going through the room scenario, you should understand the need (*as a blue teamer*) to know how to create Yara rules effectively if we rely on such tools. Commercial products, even though not perfect, will have a much richer Yara ruleset than an open-source product. Both commercial and open-source will allow you to add Yara rules to expand its capabilities further to detect threats.

If it is not clear, the reason why **file 2** was not detected is that the Yara rule was not in the Yara file used by Loki to detect the hack tool (web shell) even though its the hack tool has been around for years and has even been attributed to at least 1 nation-state. The Yara rule is present in the commercial variant of Loki, which is Thor.